

Overview and Introduction

Data Structures and Algorithms

NEU 2026

Course Staff

Professor Shlomo Geva

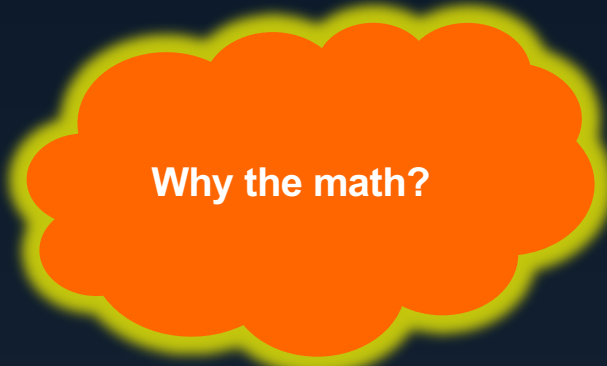
Professor Eric Ohana

Course Details

- 48 Lecture hrs
- 16 Practical hrs
- Assessment
 - Classroom work 20%
 - Laboratory homework 20%
 - Final exam 60%
- Prerequisites
 - Discrete Math
 - Java Coding skills from earlier courses

Background Knowledge

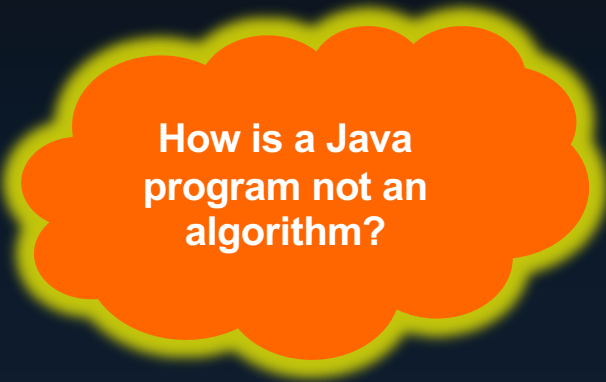
- Math/Computing
 - Logarithms
 - Basic algebra
- Graphs
- Trees

An orange, cloud-like shape with a yellow glow, containing the text "Why the math?".

Why the math?

Background knowledge

- Control flow (loops; if-then)
- Arrays
- Methods
- Classes and Objects
- Inheritance
- Exceptions
- Data Structures

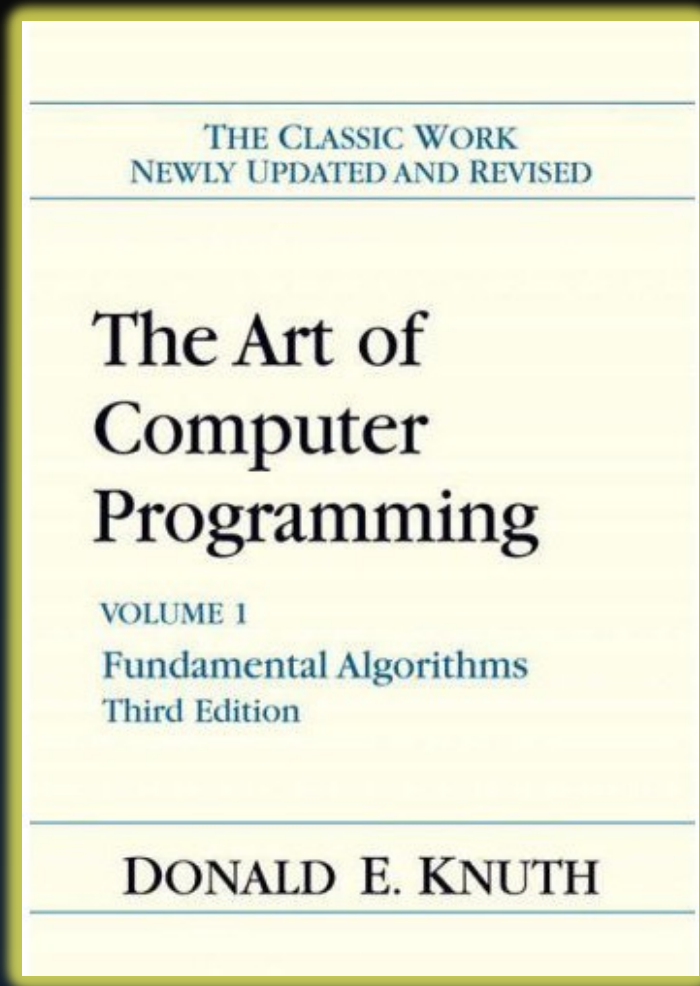


How is a Java
program not an
algorithm?

Background Knowledge

- Larger-scale programming issues
- Inter-object cooperation
- Unit testing
- Java programming language
- Java IDE (a few to choose from)

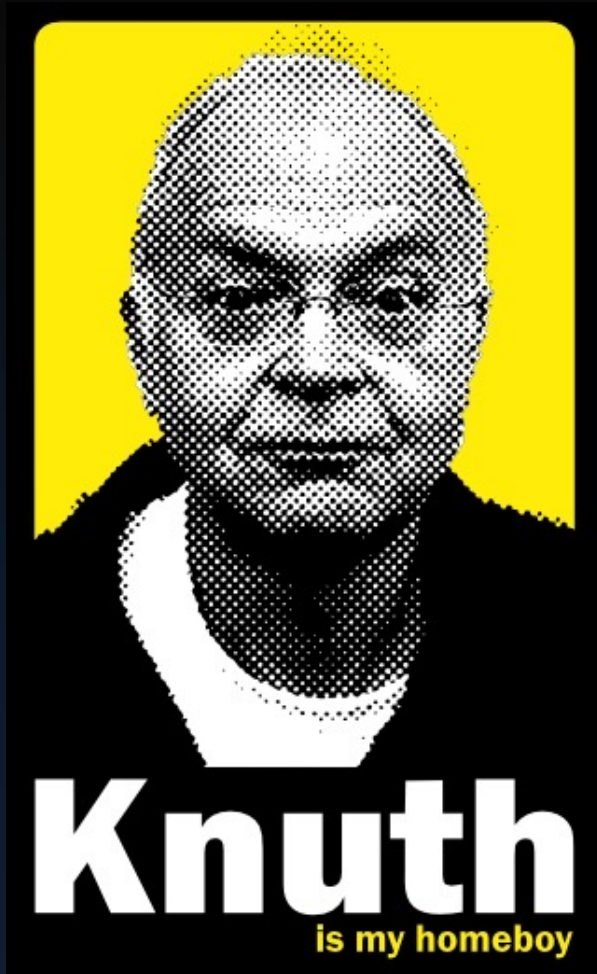
The Art of Computer Programming



“If you think you’re a really good programmer [...] read (Knuth’s) *Art of Computer Programming* [...] You should definitely send me a résumé if you can read the whole thing.”

– Bill Gates

The Art of Computer Programming



- This course tries to teach you some of this “art”
- How to think about problems algorithmically
- Problem → Algorithm → Program

Course Aims

Students should develop an understanding of

1. How data structures impact performance of complex systems
2. **Fundamental algorithms and data-structures, including their time and space requirements**
3. How to **select appropriate data structures and algorithms** to solve practical problems
4. Mathematical techniques for designing and analyzing algorithms with performance guarantees

Learning Objectives

After successfully completing this course you should be able to:

1. Understand the internal workings of fundamental data structures and algorithms
2. Determine the running time and memory space usage of common algorithms
3. Adapt or invent new algorithms and data structures for software engineering problems
4. Analyze the performance of algorithms built on fundamental data structures
5. Select and justify appropriate combinations of data structures and algorithms to solve software engineering problems

Assessment

Assessment	Due	Weighting
Exercises	During Labs	20%
homework	After weekend	20%
Final Exam	TBA Exam Period	60%

Assessment

- Practicals –
 - Demonstrate understanding of abstract data types, simple data structures, and their implementation details
 - Solve specific problems, using and implementing appropriate algorithms and data structures
- Final exam (60%)
 - Short answer and problem-solving questions
 - Covers all course material

Plagiarism

- **Definition of Plagiarism:** Copying or closely paraphrasing another person's words, data, code, or ideas without proper citation and attribution.

Plagiarism and Piracy: Copying text, major academic viewpoints, experimental data, or research results from others without clear acknowledgment.

- **Data Falsification & Fabrication:** Intentional invention, tampering, or misrepresentation of data, results, or citations.
- **Contract Cheating:** Purchasing, selling, or utilizing ghostwriters for assignments and degree theses.

Plagiarism is Serious

- If you have read something related - *reference it*.
- If you copy something - “quote it” and *reference it*.
- If you do not reference or quote when necessary *you have plagiarised* it.
- Excessive referencing or quoting is not plagiarism but may not demonstrate any understanding on your part.
- If in doubt, *ask*.

Individual Assignments – Rules of Thumb

- Don't show someone else your code
- Don't tell someone how to do the assignment

We check for excessive collaboration/plagiarism using sophisticated tools.

This Course



Is this what you thought
you signed up for?

- Considers the fundamental **data structures and algorithms** that form the basis of large, complex software systems
- Builds an understanding of the storage and processing mechanisms of computers and how to **manage storage and processing** in order to create efficient programs
- Instills knowledge to guide the selection of an appropriate structure for large, complicated systems

Overarching Aim

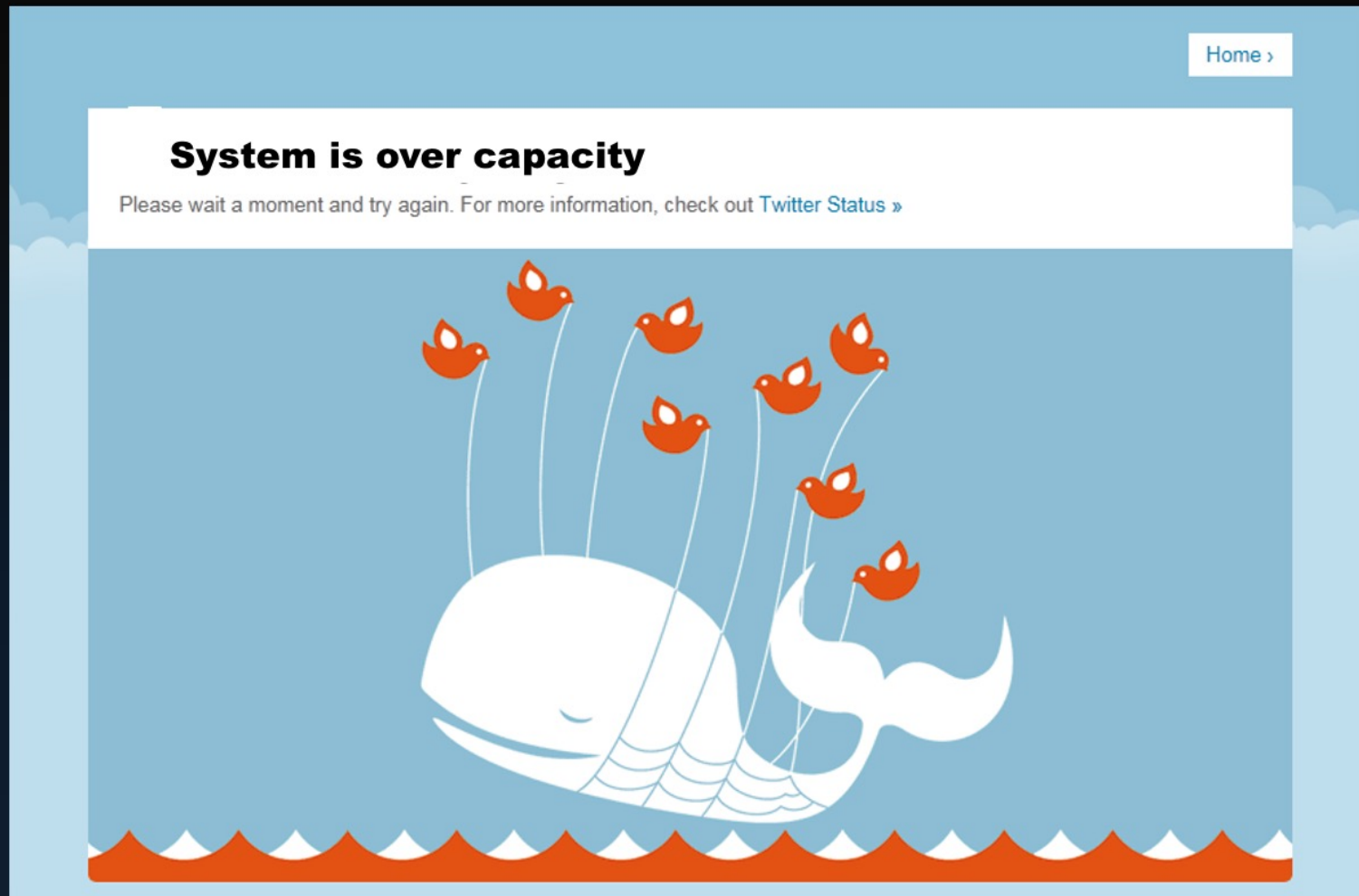
The goal of this course is to build an understanding of **data structures** and their associated **algorithms** so that you can select an **appropriate** set when designing software.

But Why?

Why do we care so much about scalability?

Throwing more machines at a problem is often cheaper than spending precious developer time.

But Why?



Motivation

- Why study algorithms & data structures?
 - Algorithms and data-structures are **fundamental** to the software in computational devices
 - Intelligent use of new and existing data structures and algorithms is a **competitive advantage** for top technology companies

Example: Web Search

- Algorithms
 - Crawl the web (web crawlers)
 - Process web pages for keywords
 - “Page-rank” algorithm to rank pages
 - Much more advanced search algorithms (e.g. LLM, AI)
- Data structures
 - Cache web pages
 - Store keywords and page URLs
 - Indices of keywords, URLs and cached site content

Example: Bioinformatics

- High throughput technologies generates enormous quantities of genomics/proteomics data
- Data integration and automated analysis issues
- Storage and management issues (disk)
- Processing is extremely demanding on CPU and memory (RAM)

Example: CERN Data Centre

- > 500 Petabytes permanent storage
- > 12 Petabytes processing memory
- Large Hadron Collider
 - generates 1 Petabyte of data per second
 - not only source of data at CERN
- Data must be filtered
 - efficiently

CERN Data Centre

- Uses tiered grid computing
 - near real-time data access
- Main DC has > 100,000 processing cores
 - 1 Petabyte data processed per day
 - 1M jobs per day
 - 6000 DB changes per second
- Grid
 - > 170 sites
 - > 2M jobs per day
 - up to 10 Gigabyte data transferred per second

Example: Databases

- How does Oracle maintain its position when its competitors provide their offerings for free?
 - Optimisation
 - Minimise query response time
 - Reliability
 - Maximise server uptime
 - Redundancy
 - Develop support for database clusters
 - Security
 - Authentication, Access Control, etc.



What You'll Learn

- Commonly used data structures and algorithms
 - What will employers expect you to know?
- How do you compare them?
- How to adapt or create algorithms to solve a new problems

ADT Definition

- Formal model of a data structure that specifies
 - **type of data** stored
 - **operations** supported
 - **types of parameters** of the operations
- Specifies *what* each operation does
 - not *how* it does it

Purpose

- Behavioural contract with user
- Encapsulation
 - Implementation details are hidden
- Isolation
 - User code and ADT implementation can both be changed without affecting each other
- Flexibility
 - Alternative implementations are interchangeable
 - Optimised for specific purpose

ADTs in Java

- **Interface** specifies **abstract methods** that an implementing class **must** implement
- Interfaces may **extend** other interfaces to add functionality

ADT Analogy – car specifications

- **GL (base model)**
 - Std Petrol engine, 5-seater, 17” wheels, 8” console, Cloth seat covers, standard colors, (other standard vehicle specs)...
- **GLI extends GL**
 - 1600cc Petrol engine, 10” console
- **GLX extends GLI**
 - 18” wheels, 4WD, Cruise control, metallic paint colors
- **GLY extends GLX**
 - Leather seats, 12” console, Bose sound system, turbo 1600cc engine, 4WD
- **GLZ extends GLY**
 - Electric vehicle, AWD
- **GLD extends GLY**
 - Turbo Diesel, 7 seater

Stack ADT

```
1 package neu.softwareSchool.adt;
2
3 import java.util.NoSuchElementException;
4
5 public interface Stack<T> {
6     public void    push(T element);
7     public T       pop() throws NoSuchElementException;
8     public T       top() throws NoSuchElementException;
9     public int     size();
10    public boolean  isEmpty();
11 }
12
13 // note the use of a template.  Different data types, even
14 // other ADTs, can be used with this generic stack
```

Using Stack ADT

```
1  public class Example {  
2  
3      // example:using a stack to store strings  
4      public static void main(String[] args) {  
5          Stack<String> stack = new StackImpl<>();  
6          stack.push("A String");  
7          stack.push("Another String");  
8  
9          String str = stack.pop();  
10     }  
}
```



Concrete Data Type

ADTs Covered

- Linear Structures
 - Stack (FILO)
 - Queue (FIFO)
 - Deque (Double-ended)
 - List
 - Sequence
- Tree
- Heap
- Priority Queue
- Hash-Table
- Map / Dictionary
- Set
- String
- Graph